Budgeting Web App – Product & Development Roadmap

Prepared for Product Management Review

Project Overview

This document outlines the product vision, system architecture, and development roadmap for a web-based budgeting application. The goal of the project is to build a modern, lightweight personal finance tool that allows users to track accounts, transactions, recurring expenses, and long■term spending projections. The system is designed to be deployable as a serverless application with minimal backend infrastructure. The frontend will run as a modern single-page application and communicate with a MongoDB database via secure serverless endpoints. A key development decision for this project is to begin with a temporary mock authentication system using a simple username/password stored locally. This allows the majority of the application to be built and tested before introducing production authentication. Real authentication (email/password or OAuth) will be implemented in the final phase of development.

Primary Product Goals

- Create a modern budgeting dashboard with projections and expense tracking.
- Allow users to manage financial accounts and transaction histories.
- Support recurring expense rules and future balance forecasting.
- Enable CSV transaction imports and potential bank integrations.
- Deploy as a serverless application that can be hosted easily.
- Design the architecture so authentication can be upgraded later without refactoring the core system.

System Architecture Overview

The application follows a lightweight serverless architecture designed for simplicity, scalability, and ease of deployment. Frontend: The frontend will be built using a modern React-based framework and styled with TailwindCSS. It will handle user interactions, data visualization, and lightweight budget calculations. Backend Layer: Instead of a traditional server, serverless API routes will handle database communication, transaction imports, and scheduled tasks. Database: MongoDB will store all user data including accounts, transactions, recurring rules, budgets, and reporting snapshots. Deployment: The application will be deployed as a serverless web app, allowing easy hosting, automatic scaling, and minimal operational overhead.

Development Roadmap

Phase 0 – Project Setup & Infrastructure

- Initialize project repository and configure development environment.
- Set up frontend framework and TailwindCSS styling.
- Create MongoDB database and initial collections.
- Deploy initial application to serverless hosting platform.
- Create basic API routes for database connectivity.

Phase 1 – Mock Authentication (Temporary System)

- Implement a simple login page using username and password.
- Store mock user session locally in the browser.
- Create a reusable user context hook for accessing the logged-in user.
- Protect application routes using the mock authentication session.
- Ensure all database records include a userId field for future compatibility.

Phase 2 – Core Database Models

- Define schema structures for users, accounts, transactions, rules, budgets, and snapshots.
- Implement validation logic for data storage.
- Create CRUD API routes for each data model.
- Ensure database indexes are optimized for queries.

Phase 3 – Dashboard UI Development

- Build the main dashboard interface.
- Display current balances and projected balances.
- Implement monthly spending summaries.
- Add category breakdowns for expenses.
- Implement navigation pages for accounts, transactions, rules, and budgets.

Phase 4 – Budget Calculation Engine

- Implement client-side calculation engine for financial projections.

- Apply recurring rules to forecast future transactions.
- Calculate projected account balances.
- Update dashboard visualizations in real time as user data changes.

Phase 5 – Transaction Import System

- Implement CSV file upload functionality.
- Parse transaction data from uploaded files.
- Provide preview and confirmation before importing transactions.
- Store imported transactions in the database.

Phase 6 – Reporting & Snapshot System

- Create aggregated monthly financial snapshots.
- Store summary statistics for faster reporting.
- Implement charts and historical trend graphs.
- Optimize queries for large transaction datasets.

Phase 7 – Bank Integration (Optional Future Phase)

- Integrate third-party financial data providers.
- Allow automatic syncing of transactions.
- Implement reconciliation tools for imported transactions.

Phase 8 – Production Authentication (Final Phase)

- Replace mock authentication with a production authentication system.
- Implement secure email/password login.
- Optionally add OAuth providers such as Google or GitHub.
- Migrate existing mock user data to the new authentication system.
- Ensure secure session management and account protection.